PART 0 · CRYPTO FUNDAMENTALS

A How the encryption works

A compact primer on the three building blocks behind this report, so that the following analysis is readable even without prior crypto knowledge. Deliberately kept conceptual.

0.1 A vault within a vault

Confidentiality rests on **two** mutually independent layers. On the outside sits the familiar **TLS** (the "https" of transport encryption). Inside TLS, the actual content is **once more** encrypted in an application-specific layer called **Application-Level Encryption (ALE)**. Anyone who opens only the outer layer still finds a sealed box inside — and the plaintext only comes into being *inside* the app.



Consequence: A classic capture of the network traffic (TLS-MITM) yields only the inner, encrypted container — not the reading.

0.2 Two methods: secrecy and authenticity

0.2.1 AES-GCM — encrypting (confidentiality *and* integrity)

A **symmetric** method: the same key encrypts and decrypts. Besides the ciphertext, "GCM" produces a 16-byte **seal** (auth tag). On decryption the seal is recomputed. If it does not match the key, nonce and data exactly, **the operation aborts and returns nothing**. Tampering is thereby ruled out. A short one-time number (the **nonce**, 12 bytes) ensures that the same message never looks the same twice.

Plaintext + Nonce (12 B) + Key → AES-GCM → Ciphertext + Auth tag (16 B)

Decryption is the reverse path. If the tag does not match, there is no plaintext.

0.2.2 RSA-PSS — signing (authenticity)

An **asymmetric** method with a key pair: the **private** key *signs*, the **public** *verifies*. What is signed is not the whole message but its hash plus a random salt ("PSS"). As a result every signature looks different even though it attests to the same message. RSA-PSS **encrypts nothing**. It only proves that a message is genuine and unaltered. AES provides secrecy, RSA provides authenticity. *Where the public and private keys for this come from is clarified in the report (§7).*

0.3 The vault around it: white-box cryptography

A **white-box** (technical name "Secure Key Box") computes with keys without ever exposing them as a readable value. The key is "baked into" the computation logic. This is the central hurdle: you cannot simply fish it out of memory, because it never appears there as a contiguous value.

0.4 And the keys themselves?

How the actual session keys arise from these building blocks — and why this works differently than one might first assume — is the **central insight** of the report. It is derived step by step (§7) and deliberately not anticipated here. With these basic concepts in hand, it is worth looking into the details: how the encryption could be analyzed despite an aggressive tamper protection, and how the chain could in the end even be fully reproduced in a separate process.

B LibreLinkUp Application-Level Encryption a reverse-engineering writeup

*A technical field report on the crypto and protection architecture of the LibreLinkUp Android app and the ways to make one's **own** glucose value usable on one's **own** smartwatch.*

Author: IamDiesel · **Date:** September 2026 · **Version:** 1.0 · **License:** CC BY 4.0

Status: Goal achieved (two independent methods, one of them verified offline). Throughout on an **unrooted** device. **Focus:** understanding & methodology. This public version is deliberately an *insights report*, not a how-to recipe.

> ⚠ **Medical disclaimer.** This is a technical security and reverse-engineering report, **not a medical document**. The methods described, and any glucose values (CGM data) obtained with them, must **under no circumstances** be used for medical decisions — in particular not for insulin dosing or other therapy adjustments. Self-built read-out paths can deliver faulty, delayed, outdated or incomplete values. For therapy decisions, only the manufacturer-approved, certified display devices and apps as well as medical advice are authoritative.

Scope, ethics & deliberate omissions

The approach described concerned exclusively one's **own** app installation, one's **own** credentials and one's **own** health data on a device in one's own possession. The motivation was personal interoperability: seeing one's own value on one's own watch.

It was important to me to carry out the entire process on an **unrooted** device in everyday condition. Without root, without a custom ROM, without defeating the platform's security. This keeps the device in its normal, trusted state (no loss of warranty, no weakened Android security) and matches the realistic setting of an ordinary user. At the same time it is the **higher hurdle**: many common hook and instrumentation approaches require root; the path described here deliberately does without.

This publication is intended as a **security / RE research writeup**. It explains architecture, concepts and methodology so that others can follow the *understanding*. It **deliberately does not** contain:

- runnable exploit / hook source code,
- the concrete Smali patches that neutralize the app's tamper protection,
- build-specific memory addresses (offsets) and crypto constants, and
- key material or personal data of any kind.

These building blocks would turn the text into a turnkey tool for circumventing the protection of someone else's app — that is not the goal. Anyone taking the same path for their own data must work out the device-specific details themselves. That is precisely part of the legitimacy threshold.

1. The problem in one sentence

LibreLinkUp encrypts its API payload **twice**: once via TLS (the normal HTTPS transport) and, on top of that, once more **inside the app** (Application-Level Encryption, "ALE"). Even someone who breaks TLS therefore sees only an encrypted container and not the glucose value.

2. System overview

Building block	Role
The app (org.nativescript.LibreLinkUp)	"Follower" app; app logic in JavaScript (NativeScript), native substructure.
A native crypto bridge (written in Go)	Encapsulates the actual encryption; contains white-box cryptography (Secure Key Box).
A native protection library (RASP)	Runtime Application Self-Protection. Detects tampering/debugger/root and makes the app crash deliberately.
Commercial obfuscator (DexGuard)	Obfuscates the Java/Smali code and brings the integrity checks with it.

Key point: confidentiality does **not** hang on TLS but on the ALE layer — and that in turn on the white-box in the Go bridge.

3. The ALE layer

Every encrypted body conceptually has the form:

```
{ "data": "<IV>.<Ciphertext>.<Auth-Tag>",   // base64, dot-separated
  "signature": "<RSA signature over data>" }
```

- **AES-GCM-128** provides confidentiality *and* integrity (12-byte nonce, 16-byte auth tag).
- **RSA-PSS** signs the data part (authenticity).

For context: the **dot-separated composition** IV.Ciphertext.Auth-Tag is an **app-specific packaging** by LibreLinkUp. **Not** a cryptographic paradigm. By default AES-GCM produces only ciphertext and an auth tag. How these parts are serialized together with the IV/nonce (here base64 with a dot as separator) is purely a design decision of the app and varies from implementation to implementation.

A classic TLS man-in-the-middle therefore yields only this container — the reason why an on-device approach is necessary.

The overall architecture, and where decryption happens, is summarized in Figure 1:

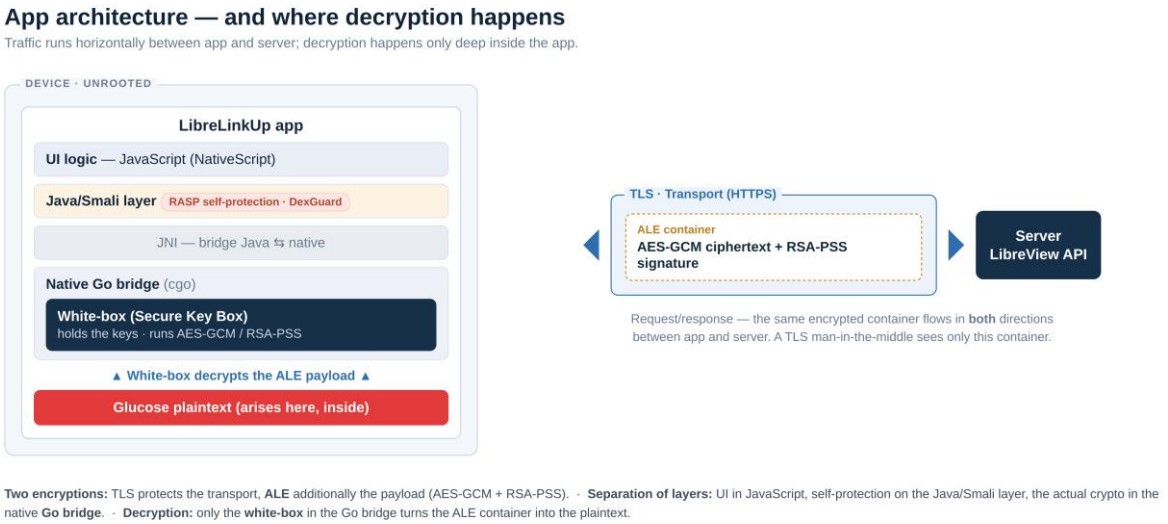


Figure 1: App architecture — and where decryption happens.
App architecture: UI in JavaScript (NativeScript), below it the Java/Smali layer with the RASP self-protection. Via JNI the native Go bridge with the white-box (Secure Key Box). TLS wraps the ALE container (AES-GCM + RSA-PSS). Only the white-box in the Go bridge decrypts the payload, so the glucose plaintext comes into being inside the app. A TLS man-in-the-middle sees only the encrypted ALE container.

4. The crypto bridge only exists at runtime

In the APK the bridge is not present as a normal, readable library but **packed/encrypted**. Only at startup does a native loader unpack it into a **temporary** file in the app's private folder, load it and **immediately delete it again** (the open file descriptor keeps it alive). Consequences:

- The APK copy is worthless for analysis. The real code exists only in the **decrypted runtime state**.
- The time window is tiny. The file grows progressively and then disappears → one has to "race" the write in order to get a complete image.
- It is precisely over this decrypted image that the tamper protection forms its **checksum** at runtime. A hook must therefore change nothing in this native code.

From the complete image the exported crypto functions can be read off via the ELF's dynamic symbol table. Note: the debug symbols are stripped, but the *dynamic* exports must remain present, otherwise the library would be unusable.

5. The protection layers (why it is hard)

- **Isolated linker namespace**: a standard symbol lookup from another library does not find the functions. Addresses have to be determined via the module base from /proc/<pid>/maps.
- **Native integrity check**: a checksum is formed over the bridge's code at runtime. Every byte patch to this code is noticed.
- **Java-layer RASP:** implemented for the lifecycle methods and the most effective protection layer: upon detected tampering the app reacts deliberately destabilizingly (random exceptions on several threads, a warning message, memory-allocation bombs) and terminates itself **before** the login.
- **Dynamically registered JNI methods:** the protection library's methods are registered dynamically, so they cannot simply be replaced by a dummy.
- **Frida detection**: known instrumentation signatures are detected in memory.

Rule of thumb: the app stays stable only if one addresses **both** layers. Do *not* alter the native code **and** defuse the Java RASP.

Roadmap for the next chapters: first the direct path to the plaintext (**Method 1**). Derived from it, the **key model** and, building on that, the standalone path without a running app (**Method 2**).

6. Method 1 — tapping the plaintext (live hook)

No matter how well the key is protected: the decryption function **must** at some point write the plaintext into an output buffer. Whoever reads this buffer after the call has the value, without knowing the key.

The first, naive way is to overwrite the start of the function with a jump. It did work and delivered real plaintext, but it was detected by the integrity check and led to a crash. It does, after all, change bytes in the protected code.

The **stable** way uses **hardware breakpoints**: the CPU's debug registers fire *before* an instruction runs — and that **without changing a single byte of the protected crypto code**. The checksum over this code thus stays "clean". Because a process may not observe itself this way, a tiny observer process started alongside it takes over setting the breakpoints and, after the call, reads the decrypted buffer out of the app's memory.

Is the app then not modified at all?

The app itself very much is modified — the observer library is loaded into the app via a bytecode change (Smali patch), and further patches defuse the self-protection checks. The point about the hardware breakpoints is narrower: only the **checksum-protected native crypto code** stays untouched. That is exactly the part the integrity check monitors.

Insight: against runtime integrity checks a *byte-preserving* observation hook (hardware breakpoint) beats the classic *byte-altering* inline hook.

7. The key model (the central insight)

This chapter builds directly on Method 1: the same hardware breakpoint that reads out the plaintext there can also record the **arguments** of the crypto calls — and it is precisely from these live observations that the following key model emerges. It is at the same time the basis for how, in Method 2, one can work entirely without the running app.

The intuitive model — "the AES key is renegotiated per session via Diffie-Hellman (ECDH)" — **is not correct for ongoing operation**. In **normal operation nothing is negotiated**: the session keys arise deterministically from **static material embedded in the library** plus a **package delivered per session by the server** (as shown in a moment).

How does one even obtain the session package?

The way *there* was examined too — not just the actual decryption step. It runs in two steps:

1. Login: logging in with email and password yields merely an **access ticket** (technically a *JWT*). It carries one's own account/installation identity but **no** key material.

2. Provisioning: a subsequent request fetches the **per-session package**. It arrives **encrypted in the server response** and is readable solely with the **static, installation-independent** base building block. This one request is encrypted with **AES-GCM** and signed with **RSA-PSS** — the same two methods from Part 0 and §3, here in action.

And here lies the real crux of the standalone path: the server does **not deliver this package on mere request**. The client must **actively register itself as a device with the backend**. A passive replay of an intercepted call is not enough and runs into nothing (the server then responds only with an empty reaction). It is precisely this **self-registration** that was the non-obvious hurdle which makes the standalone client work in the first place. The concrete procedure behind it is deliberately left out here.

This answers the initially open question of where the session material comes from: **from the server response**, not from a local ECDH handshake.

And during ongoing operation?

Once the session is provisioned — including the self-registration just described — the original app is no longer needed: from then on a standalone client for the ongoing retrieval makes do **with email and password alone**, i.e. without any on-device capture. The actual hurdle therefore lies **once** in the provisioning, not in normal operation.

How is one authenticated after login?

The password travels over the wire exactly **once**: at login. After that every call identifies itself via the **access ticket (JWT)**: the server signed it itself and recognizes it again by its own signature. It thus knows that the call belongs to this account — entirely without a renewed password.

To be distinguished from this is the **app/device authenticity** via **RSA-PSS**. Here a precise distinction is worthwhile, because there are **two signature situations at two points in time**. What is signed is exclusively the **one provisioning request** — and that already with the **static pre-session material**. So *before* any session keys even exist. This signature therefore serves as proof of a "genuine app" that lets the server hand out the session package. The actual glucose retrievals afterwards are plain requests **authenticated** with the ticket **entirely without a signature of their own**; only their payload is additionally **AES-GCM**-encrypted. The "signing key" derivable from the key ladder (in a moment) is therefore not needed at all for pure reading.

And the asymmetric (public/private) keys?

Because RSA-PSS is asymmetric, the question of the key pair arises. For reading/normal operation the answer is lean:

- **Signing (app side):** the provisioning request is signed with the **static material embedded in the library**. **No** fresh private key is generated per session.
- **Verifying (server responses):** the **public key of the server** needed for verification is **delivered within the provisioning package**. A separate key exchange over a further channel is not necessary.
- **Initial setup:** an actual generation or negotiation of a device key pair belongs, if at all, to the **one-time account/device setup.** That is not a primary subject of this investigation.

The following flow (Figure 2) summarizes login, provisioning and retrieval:

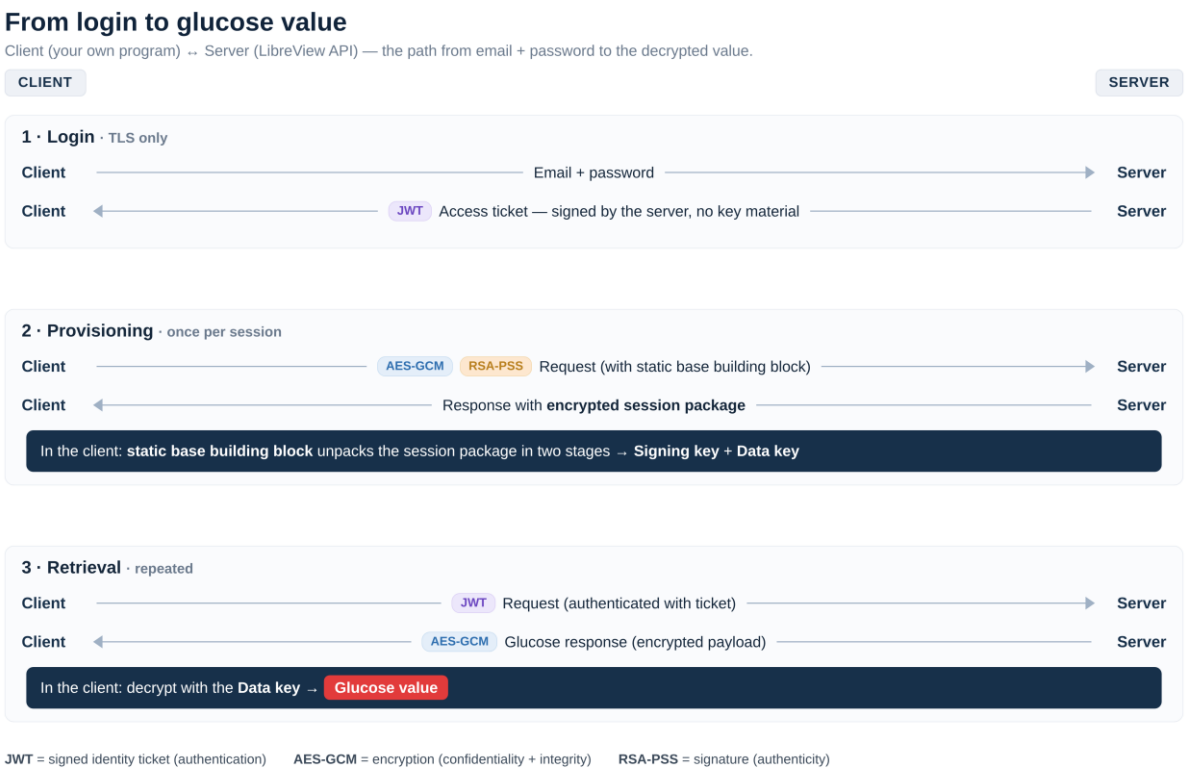


Figure 2: From login to glucose value.
Flow: Login → access ticket (JWT); provisioning (AES-GCM-encrypted + RSA-PSS-signed) → encrypted session package → unpack with the static base building block into signing and data keys; then repeated retrieval with the ticket → AES-GCM-encrypted glucose response.

If one looks more closely at *how* a usable key arises from the session package, the actual model becomes visible: the **two-stage key ladder**. The **static base building block** from the diagram does double duty here: it already decrypts the ongoing data *and* it **opens the session package.** Thus it starts the ladder. From the package, then, in **two unwrap stages** two further keys are "unwrapped" one after another.

Together these are **three roles**:

Role	Origin	Used for
Bootstrap key	the static, embedded base building block itself	Decrypts config data and request bodies, opens the session package (start of the ladder) and signs the provisioning request (pre-session, see above)
Signing key	Stage 1 of the unwrap (from the session package)	Available for request signatures. But not needed for the pure glucose retrieval (the retrievals carry only the ticket)
Data key	Stage 2 of the unwrap (from the session package)	Decrypts the payload — among other things the glucose value

In short: **one** hard-wired building block starts the chain, and out of the server package the signing and data keys fall one after another. For the pure glucose value the **second stage** suffices in the end. "The key" is thus neither a single static value nor a freely negotiated secret, but the result of this deterministic unpacking: **static building block + session package → session key**.

Where does this model come from?

The key ladder itself is not a given but **derived from a runtime trace**. Via a hardware breakpoint on the entry of the unwrap and decryption functions, the arguments were read out on every call according to the ARM64 calling convention. *As an aside, **AAPCS64**: the first arguments sit in fixed CPU registers, further ones on the stack. So one knows which parameter lies in which register.*

Ultimately three observations led to the model:

- **Two stages**: the unwrap function fired **twice** per login with different input material. The output of the first run demonstrably fed the second.
- **Static vs. per-session**: one set of input blobs stayed **bit-identical** across all logins and devices (they are embedded firmly in the library). The others **varied with every login** (those are the server-delivered bundles). It is exactly this difference that separates bootstrap material from session material.
- **Role assignment:** which unwrapped key has which role was confirmed via **object identity**. It was examined which key subsequently feeds which operation (signing vs. decrypting) and via the **content of the respective plaintext produced**.

The ladder is thus not an axiomatic statement but a finding evidenced by runtime behavior.

8. Method 2 — the white-box as a service provider (standalone oracle)

Building on the key model (§7), the decryption can now be reconstructed entirely without the running app.

Instead of tapping it in the running app, one can load the decrypted crypto bridge into a **separate** process and call the crypto functions oneself. One thus uses the crypto bridge as an "oracle" — and the key never has to be extracted for this.

What is "Replay-Mint" — and why can one not simply copy?

The internal crypto objects (engine, data objects, session keys) are **living C++ objects** on the **heap** (the memory area for dynamically created objects): they contain pointers to further objects and a **vtable** (the table of their methods). If you pull them raw out of a memory dump and load them into another process, all these pointers point into the void and the copy crashes immediately. This is the **"vtable-object wall"**.

The way out is to **not copy the objects but to mint them anew** (*to mint*): one replays the **exact initialization order** of the real app (*replay*) and lets the white-box engine freshly create the same objects in one's **own** process. Because they thus come into being in one's own address space, the vtable and heap pointers are automatically valid.

This order is quite concrete and no abstract ritual: first the central **engine object** of the white-box is created. Then the **embedded static blobs** are imported as its data objects (so the engine "knows" its hard-wired key raw material). Only on top of that do the **unpacking and decryption steps** build. It is exactly the chain the app runs through itself on every startup — just redone in one's own process. This combination — replaying the order **and** minting objects anew — bears the name **"Replay-Mint"**.

Why does this work in a foreign process?

The decisive technical lever: the crypto bridge is written in Go (via **cgo**, the bridge through which Go code embeds native C code) and brings its **own runtime environment** with it. This **initializes itself on loading the**

library (dlopen). The same constructors that run on app startup run unchanged in one's own process too. That is why the "replaying" is no blind attempt: after loading, the engine already stands ready, and the static initialization on which all further objects build has taken care of itself.

This is not entirely free, though. Loading in one's own process succeeds only if the **C++ runtime environment** on which the Go bridge rests is available there first. If this dependency is missing, the dlopen call already aborts, before a single crypto function is reachable. If this prerequisite is met, the self-initialization kicks in. Without it the whole oracle approach would be hopeless.

How does one even find the right functions?

That the runtime initializes itself only means: the engine stands **ready**, but it does *not compute* on its own. To encrypt and decrypt, one has to call the white-box's crypto functions **oneself**: decrypt/encrypt, key-unwrap and sign, plus the internal constructors for engine and data objects (the "Replay-Mint" chain from above). But these functions must first be **found** in the library. The procedure for this is sketched roughly (and deliberately without concrete addresses), as follows:

- **Grab the image:** first secure the *decrypted* runtime image of the bridge (the APK copy is packed and unusable, see §4).
- **Read off exported functions:** a part of the functions is *exported* and stands in the **dynamic symbol table** of the ELF. This is the list of all outwardly visible functions with name and address that the dynamic linker strictly needs. This list is preserved even when the debug symbols are stripped. From it one reads the relative address directly. At runtime the load base follows from a known anchor, and every further address is then **base + fixed offset**.
- **Localize internal functions:** the *non-exported* helpers (e.g. the constructors for engine and data objects) show up in the **live trace** as caller addresses and are confirmed by disassembly.

The concrete values are tied to an app/library version and are deliberately not stated here.

But this is only half the battle. A crypto function of the white-box is no free-standing routine but a **method on a living object**. These objects cannot simply be copied over from a foreign process (the "vtable-object wall" from above). One therefore needs **two things at once**: the *addresses* of the functions (track A) and *valid objects* on which to call them (track B, freshly minted via Replay-Mint). Figure 3 summarizes this relationship:

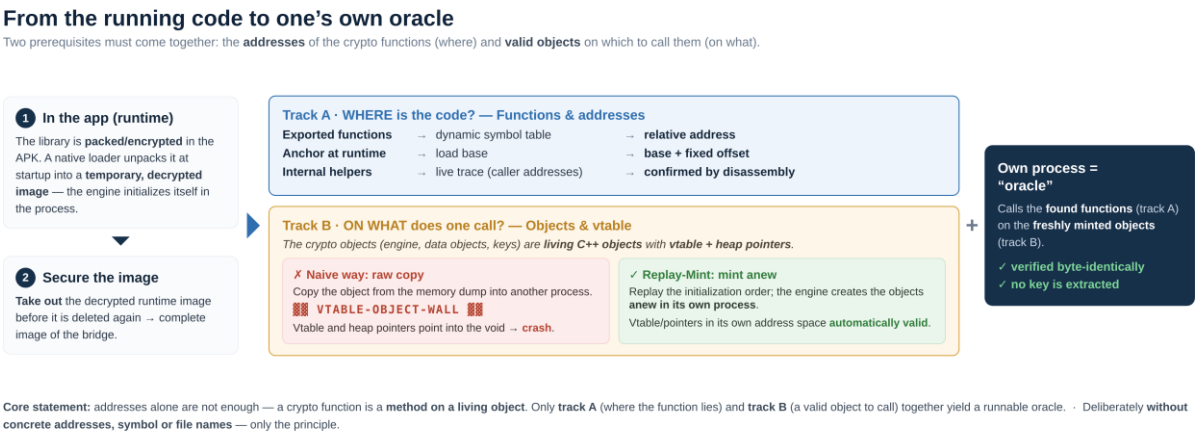


Figure 3: From the running code to one's own oracle.

From the running code to one's own oracle: in the app the packed library is unpacked at runtime into a temporary, decrypted image and secured. Then two tracks come together: track A determines the addresses of the crypto functions (exported functions via the dynamic symbol table, anchor to the load base with base-plus-offset, internal helpers via live trace and disassembly). Track B solves the object problem (the crypto objects are living C++ objects with vtable and heap pointers; raw copying fails at the vtable-object wall, whereas Replay-Mint mints the objects freshly in one's own process so that their pointers are valid). Both tracks together yield an oracle in one's own process that was verified byte-identically, without extracting a key.

This method was **verified byte-identically**: the session key reconstructed in one's own process decrypts a real server response exactly. This proves an important property: **the white-box is not process-bound**. The session

key arises deterministically from data embedded in the library plus a bundle delivered per session by the server. The **device portability** follows from the account-independence of this embedded data, but was not verified across multiple devices (finding on one device).

What happens here methodologically — and what does not?

This attack **does not break the white-box**: it attacks the mathematical key obfuscation at no point (no DCA/DFA, no key extraction). Rather, it is classic **"code lifting"**: the white-box is reused as a **black-box service provider in one's own process**. Its legitimate call interface is reused and operated exactly as the app would. The raw key stays the entire time inside the protected interior of the white-box. It is **circumvented, not cracked**. This distinction is important: the hardening of the white-box mathematics is thereby not refuted. It is merely effective in the wrong place, because the call interface is reproducible in one's own process.

Does this also work in practice?

Method 2 did not remain theoretical: it was implemented as a **runnable proof of concept**. This is a standalone client that **logs in** with email and password and **retrieves and decrypts one's own glucose value live** — entirely without the original app.

9. Overarching insights

- **Double encryption (ALE over TLS):** renders pure network MITM ineffective. The interesting point lies where the plaintext comes into being anyway.
- **Runtime integrity checks: one defeats them not through cleverer byte patches on the protected code but through pure observation without any byte change** at that very code (hardware breakpoints), while the app is indeed modified elsewhere (via a bytecode patch).
- **Multi-layer protection requires multi-layer answers**: native code *and* Java RASP have to be considered together.
- **White-box does not automatically mean process-bound**: here the chain could be reproduced deterministically offline in a foreign process. That is the genuinely surprising finding. The device portability follows from the account-independence of the embedded blobs but is so far evidenced only on one device.

10. Alternative paths (classification)

Path	RE effort	robust against forced ALE	Assessment
REST client (open follower interface)	None	No (breaks as soon as ALE is enforced server-side)	Cheapest path as long as it holds
On-device hook (Method 1)	Medium	Yes	Delivers exactly what the real app sees
White-box oracle (Method 2)	High	Yes	Patch-free, survives forced ALE; the cleanest self-built solution
Key extraction (DCA/DFA against the white-box)	Very high	Yes	Uneconomical — the oracle achieves the same without seeing the key
Read the sensor directly (Juggluco/xDrip and similar)	Low–medium	Yes (no server)	Technically strong if you wear the sensor yourself, but decoupled from LibreLinkUp/LibreView and/or shifts the upload to a third-party app (a drawback if the treating physician uses the follower service)

11. Data protection

All decrypted data and all key material are personal health data and remain local on one's own device. In this publication neither real keys, UUIDs or email addresses nor build-specific addresses appear.

This report documents a personal interoperability solution and the insights gained along the way about app hardening and white-box cryptography. It is not a call to action and not a guide to attacking someone else's data or devices.

Appendix — Glossary

The most important technical terms of this report in brief:

Term	Meaning
TLS	Transport encryption of “https” — protects the connection to the server.
ALE (Application-Level Encryption)	Encryption of the payload at the application level, additional to and independent of TLS.
AES-GCM	Symmetric encryption method with a built-in authenticity seal (auth tag).
RSA-PSS	Asymmetric signature method; proves authenticity, encrypts nothing itself.
Nonce	One-time number that makes the same message look different on every encryption.
White-box (Secure Key Box)	Crypto method that computes with keys without ever exposing them as a readable value.
JWT (access ticket)	Server-signed credential that assigns a call to an account — without a renewed password.
RASP (Runtime Application Self-Protection)	Self-protection that detects tampering/debugger at runtime and makes the app crash deliberately.
cgo	The bridge through which code written in Go embeds native C code.
Vtable	The method table of a C++ object; contains pointers to its functions.
Heap	The memory area for dynamically created objects; their addresses are valid only within one’s own process.
ELF	The file format of executable programs and libraries on Linux/Android.
Dynamic symbol table	The list of outwardly visible functions (name + address) remaining in the ELF, which the linker needs.
AAPCS64	The ARM64 calling convention: which function arguments stand in which CPU registers.
Hardware breakpoint	CPU-assisted halt at an address — observes code without changing it.
Code Lifting	Reusing a legitimate crypto interface in one’s own process, without extracting the key.